

MLIR / NPU Compiler Course - Lecture 05

Dialect System: Frontend / Graph / Kernel / Runtime Dialect 설계

1. 강의 개요

5강은 MLIR의 확장성 핵심인 Dialect 시스템을 학습한다. NPU compiler에서는 하나의 IR로 모든 단계를 표현하려고 하면 model semantics, graph optimization, kernel scheduling, runtime ABI가 뒤섞인다. Dialect는 이 복잡도를 abstraction level 별로 나누고, 각 단계의 legality와 verification boundary를 명시하는 장치다.

- 강의 주제: Dialect 시스템
- 핵심 실습/산출물: frontend / graph / kernel / runtime dialect 분리 설계
- 선수 지식: 1-4강의 MLIR 큰 그림, Operation/Value/Type/Attribute, Region/Block, mlir-opt/FileCheck
- 최종 목표: custom NPU dialect를 설계하기 위한 namespace, operation, type, attribute, verifier, lowering boundary 초안 작성

2. Learning Objectives

- Dialect가 MLIR extension mechanism에서 어떤 역할을 하는지 설명한다.
- operation, type, attribute, trait, interface, verifier, parser/printer가 dialect 안에서 어떻게 연결되는지 읽는다.
- AI/NPU compiler에서 frontend, graph, kernel, runtime dialect의 책임을 분리한다.
- npu.matmul, npu.dma.load, npu.barrier, npu.command_buffer 같은 pseudo op의 dialect 소속과 verification rule을 설계한다.
- ODS/TableGen 기반 dialect 정의 파일의 큰 구조를 이해하고, 실전 구현 전의 설계 spec을 작성한다.

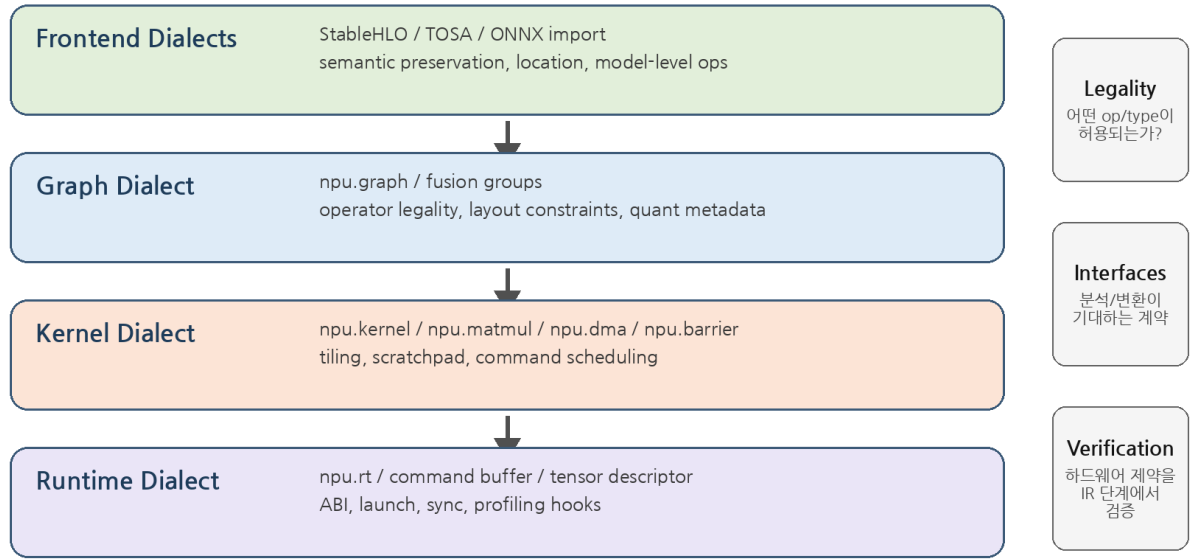
3. 왜 NPU Compiler에서 Dialect가 중요한가?

NPU compiler는 일반 CPU compiler보다 더 많은 target-specific constraint를 조기에 다뤄야 한다. 예를 들어 tensor shape는 모델 semantics에 가깝지만, tile shape는 SRAM capacity와 DMA alignment에 가깝다. quantization scale은 graph-level transformation에 필요하지만, accumulator width와 requant shift는 kernel/runtime에도 영향을 준다. Dialect를 잘 나누면 각각의 정보가 가장 자연스러운 IR level에서 검증된다.

- Frontend level: StableHLO/TOSA/ONNX import에서 framework semantics와 source location을 보존한다.
- Graph level: fusion group, layout propagation, quantization metadata, op legalization을 표현한다.
- Kernel level: tile loop, DMA, scratchpad, accumulator, barrier, vector/tensorization mapping을 표현한다.
- Runtime level: command buffer, tensor descriptor, launch parameter, stream/sync, profiling/event를 표현한다.

MLIR Dialect Layering for AI/NPU Compiler

각 dialect는 서로 다른 abstraction level과 correctness boundary를 갖는다.



4. Dialect 의 기본 개념

MLIR에서 dialect는 이름공간(namespace)을 갖는 IR extension 단위다. dialect는 operation, type, attribute를 정의하며, 필요하면 operation interface, trait, custom verifier, custom parser/printer, canonicalization pattern, lowering pass와 함께 사용된다. 중요한 점은 dialect가 “문법 묶음”이 아니라 compiler pass들이 공유하는 semantic contract라는 것이다.

구성 요소	역할	NPU 예시
Namespace	operation/type/attribute 이름의 소속을 구분	npu.matmul, !npu.tile, #npu.layout
Operation	IR에서 실행/계산/제어/메타 동작 표현	npu.dma.load, npu.matmul, npu.barrier
Type	값의 compile-time shape/layout/handle 계약	!npu.tensor_desc, !npu.command_buffer
Attribute	compile-time metadata 및 고정 파라미터	tile_shape=[16,16], memory_space=sram
Trait	operation의 구조적 성질 선언	Pure, SameOperandsAndResultType, SingleBlock
Interface	pass가 공통으로 호출할 수 있는 semantic API	MemoryEffectOpInterface, TilingInterface
Verifier	IR invariants와 target legality 검사	DMA alignment, SRAM footprint, accumulator width
Parser/Printer	읽기 쉬운 assembly syntax 제공	npu.matmul %a, %b into %acc {tile=[16,16,32]}

5. Operation Naming 과 Namespace 원칙

operation 이름은 보통 dialect namespace와 mnemonic을 합쳐 dialect.op 형태로 쓴다. 예를 들어 arith.constant, func.func, linalg.matmul, memref.alloc, scf.for 처럼 이름만 봐도 abstraction level을 추정할 수 있어야 한다. NPU dialect에서도 npu_graph.fuse, npu_kernel.matmul, npu_rt.launch 처럼 역할이 보이게 설계하는 것이 좋다.

- 좋은 이름: npu_kernel.matmul - tiled kernel operation임을 바로 알 수 있다.
- 나쁜 이름: npu.op1 - pass/debug/test에서 의미를 추적하기 어렵다.
- 과도한 단일 dialect: npu.matmul이 graph op인지 kernel op인지 불명확해진다.

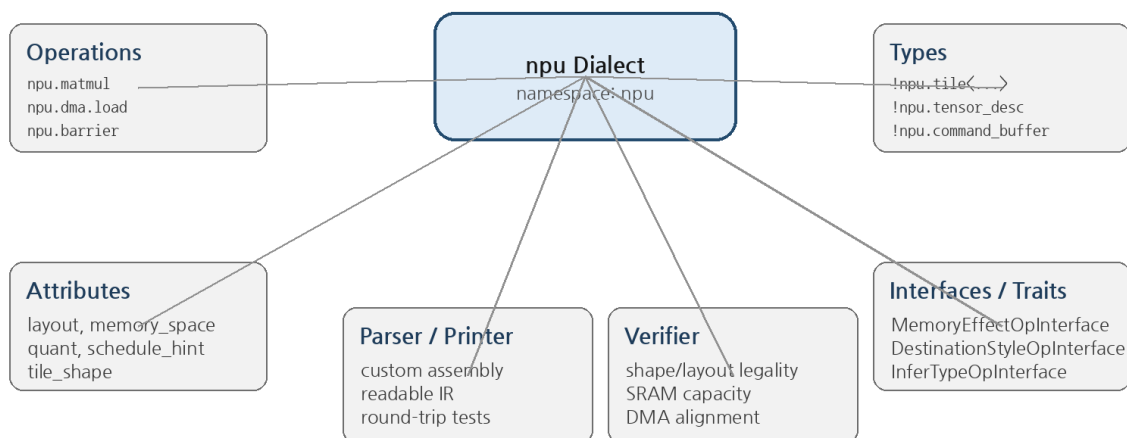
- 과도한 세분화: dialect 가 너무 많아져 conversion/pass registration 이 복잡해진다.

6. Dialect Anatomy

아래 그림은 하나의 custom NPU dialect 가 가져야 할 구성 요소를 요약한다. 실제 제품 수준의 NPU compiler 에서는 dialect 정의보다 verifier, interfaces, canonicalization, conversion target 정의가 더 많은 engineering cost 를 차지한다.

Dialect Anatomy

Dialect = namespace + operations + types + attributes + interfaces + verification + lowering boundary



NPU 설계 관점: dialect의 역할은 “예쁜 namespace”가 아니라 compiler pass와 HW legality가 공유하는 계약을 명시하는 것이다.

7. Attribute vs Operand vs Type Decision

NPU dialect 를 설계할 때 가장 자주 하는 판단은 어떤 정보를 operand, attribute, type 중 어디에 둘 것인가이다. 원칙은 간단하다. 런타임 값이거나 dataflow 에 참여하면 operand, compile-time static metadata 면 attribute, 값 자체의 ABI/shape/layout 계약이면 type 으로 둔다.

정보	권장 위치	이유
입력 activation tensor	Operand	def-use chain, alias/lifetime, bufferization 대상
Weight tensor	Operand 또는 SymbolRef	상수 fold 가능하더라도 dataflow/weight section 과 연결 필요
Tile shape	Attribute	compile-time scheduling decision
SRAM memory space	Type 또는 Attribute	memref memory space 면 type, op hint 면 attribute
Tensor descriptor handle	Type	runtime ABI 와 값의 의미가 결합
DMA stride/offset	Attribute 또는 Operand	static 이면 attribute, dynamic shape/runtime stride 면 operand
Quant scale/zero-point	Attribute 또는 side-table	static per-channel quant 는 attribute, calibration table 은 symbol/metadata 가능

8. ODS/TableGen 관점

MLIR operation 은 주로 ODS(Operation Definition Specification)와 TableGen 으로 선언한다. ODS 는 op 이름, summary, operands, results, attributes, traits, assembly format, verifier hook, builder 등을 declarative 하게 기술하는 방식이다. 이 수업에서는 완전한 buildable code 가 아니라 구조를 읽고 설계하는 데 집중한다.

```
// Educational pseudo TableGen skeleton
include "mlir/IR/OpBase.td"

def NPUDialect : Dialect {
  let name = "npu_kernel";
  let summary = "NPU kernel-level operations";
  let description = [{
    Tiled compute, DMA, barrier and scratchpad-level scheduling operations.
  }];
}

class NPUKernel_Op<string mnemonic, list<Trait> traits = []>
  : Op<NPUDialect, mnemonic, traits>;

def NPUKernel_MatMulOp : NPUKernel_Op<"matmul", []> {
  let summary = "Tiled matrix multiply on NPU array";
  let arguments = (ins AnyMemRef:$lhs, AnyMemRef:$rhs, AnyMemRef:$acc,
    I64ArrayAttr:$tile_shape,
    StrAttr:$accumulator_type);
  let results = (outs AnyMemRef:$result);
  let assemblyFormat = "$lhs `, ` $rhs `, ` $acc attr-dict `:` type($lhs) `, ` type($rhs) `->` type($result)";
  let hasVerifier = 1;
}
```

9. NPU Compiler 용 Dialect 분리 모델

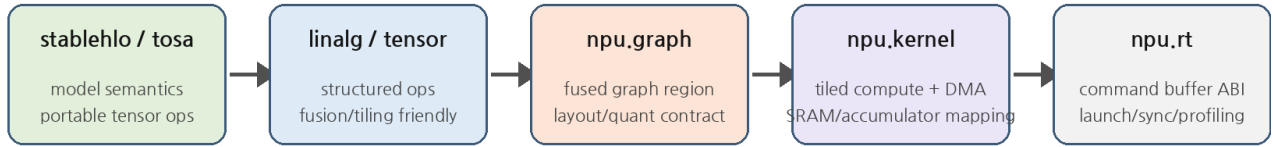
이번 강의의 핵심 산출물은 frontend/graph/kernel/runtime dialect 분리 모델이다. 아래 표는 실무 설계에서 바로 사용할 수 있는 기준이다.

Dialect Layer	주요 책임	대표 IR/Op	검증해야 할 것
Frontend	framework semantic 보존, import/export, source location 유지	stablehlo.dot_general, tosa.conv2d	모델 op semantic, shape rank, dtype legality
Graph	op fusion, layout propagation, quant metadata, partitioning	npu_graph.fusion_group, npu_graph.layout_cast	fusion legality, layout compatibility, quant granularity
Kernel	tile scheduling, DMA/barrier, scratchpad/accumulator 사용	npu_kernel.matmul, npu_kernel.dma_load	SRAM capacity, alignment, hazard, accumulator width
Runtime	command buffer ABI, descriptor, launch/sync, profiling	npu_rt.create_desc, npu_rt.launch	ABI version, descriptor size, stream/event dependency

10. Progressive Lowering 과 Dialect Boundary

MLIR 의 강점은 여러 dialect 가 한 module 안에 공존하면서 점진적으로 lowering 될 수 있다는 점이다. pass 는 특정 dialect op 를 낮추되 아직 합법적인 다른 dialect op 는 남겨둘 수 있다. NPU compiler 에서는 model graph 전체를 한 번에 command buffer 로 내리는 대신, 각 boundary 에서 legality 와 metadata 를 점진적으로 확정하는 방식을 권장한다.

Dialect Boundary and Progressive Lowering



각 boundary에서 ConversionTarget, RewritePattern, TypeConverter, Verifier가 달라진다.

실무 원칙: HW 제약은 너무 늦게 runtime error로 발견하지 말고, 가능한 한 dialect verifier/pass에서 조기에 잡는다.

11. Example: MatMul + Bias + ReLU Lowering

동일한 computation도 dialect level에 따라 의미가 달라진다. frontend level에서는 정확한 tensor semantics가 중요하고, graph level에서는 fusion과 layout이 중요하며, kernel level에서는 tile/DMA/barrier가 중요하고, runtime level에서는 command buffer ABI가 중요하다.

```
// frontend-ish
%0 = stablehlo.dot_general %x, %w, ... : tensor<1x128x256xf16>, tensor<256x64xf16> -> tensor<1x128x64xf16>
%1 = stablehlo.add %0, %bias : tensor<1x128x64xf16>
%2 = stablehlo.maximum %1, %zero : tensor<1x128x64xf16>

// graph-ish pseudo
%g = npu_graph.fusion_group @matmul_bias_relu(%x, %w, %bias)
    {layout = #npu.layout<NHWC>, quant = #npu.quant<i8, per_channel>}}

// kernel-ish pseudo
npu_kernel.region @tile_16x16x32 {
  npu_kernel.dma_load %x_tile to %sram_a {align = 64}
  npu_kernel.dma_load %w_tile to %sram_b {align = 64}
  npu_kernel.barrier
  npu_kernel.matmul %sram_a, %sram_b, %acc {tile_shape = [16, 16, 32], acc = "i32"}
  npu_kernel.requant_relu %acc to %out_tile
  npu_kernel.dma_store %out_tile to %y
}

// runtime-ish pseudo
%cb = npu_rt.create_command_buffer
npu_rt.launch %cb, %desc_x, %desc_w, %desc_y {core_mask = 0x0f, qos = "realtime"}
```

12. Verifier 설계 체크리스트

- Shape legality: matmul K dimension, convolution channel grouping, broadcasting rule 검증
- Type legality: input dtype, accumulator dtype, quantized dtype, mixed precision policy 검증
- Layout legality: tensor layout, memory layout, bank conflict 위험, vector alignment 검증
- Memory legality: SRAM footprint, double buffering 가능성, DMA stride/offset alignment 검증
- Schedule legality: barrier ordering, RAW/WAR/WAW hazard, async token dependency 검증
- Runtime legality: descriptor ABI version, command buffer size, stream/event dependency 검증

13. Pass 설계와 Dialect 의존성

Pass가 특정 dialect의 op/type/attribute를 생성한다면 해당 dialect를 MLIRContext에 로드해야 한다. 실무에서는 pass registration과 getDependentDialects 선언이 빠지면 멀티스레드 pass pipeline이나 out-of-tree compiler build에서 문제가 된다. Dialect 분리는 코드 구조, CMake target, pass pipeline, test layout과 함께 설계해야 한다.

```
// Educational C++ pseudo skeleton
struct LowerGraphToNPUKernelPass
: PassWrapper<LowerGraphToNPUKernelPass, OperationPass<ModuleOp>> {
  void getDependentDialects(DialectRegistry &registry) const override {
    registry.insert<npu::kernel::NPUKernelDialect,
                  memref::MemRefDialect,
                  scf::SCFDialect,
                  arith::ArithDialect>();
  }

  void runOnOperation() override {
    // 1. Define legal/illegal dialects or ops
    // 2. Populate rewrite patterns
    // 3. Apply partial/full conversion
  }
};
```

14. 실습 안내

실습은 build 가능한 MLIR compiler를 완성하는 것이 아니라, custom NPU compiler 설계를 위한 dialect boundary를 명확히 하는 것이 목표다. worksheet와 examples 폴더를 함께 사용한다.

- Exercise A: 주어진 op 목록을 frontend/graph/kernel/runtime dialect로 분류한다.
- Exercise B: npu_kernel.matmul operation schema를 operand/result/attribute/type/verifier 관점으로 작성한다.
- Exercise C: npu_graph.fusion_group과 npu_kernel.region의 boundary를 정의한다.
- Exercise D: ODS pseudo skeleton에 verifier rule과 assembly format 초안을 채운다.
- Exercise E: 최종 lowering pipeline에서 dialect 별 pass boundary와 FileCheck test 이름을 작성한다.

15. Common Mistakes

실수	문제	개선 방향
모든 것을 npu dialect 하나에 넣기	graph op와 kernel op의 legality가 섞임	npu_graph / npu_kernel / npu_rt로 boundary 분리
하드웨어 제약을 attribute에만 숨기기	pass가 semantic을 해석하기 어려움	type/interface/verifier로 명시화
custom op를 너무 일찍 도입	linalg/vector 최적화를 재사용하기 어려움	가능한 한 standard dialect에서 최적화 후 target op로 lowering
verifier 없이 pseudo op만 추가	잘못된 IR이 pipeline 뒤쪽에서 폭발	shape/layout/memory/schedule verifier를 조기 도입
parser/printer를 무시	test와 debug가 어려움	assemblyFormat과 FileCheck 친화적 IR 제공

16. 과제

- 과제 1: MatMul + Bias + ReLU를 대상으로 frontend/graph/kernel/runtime dialect 별 IR snapshot을 한 페이지로 작성한다.
- 과제 2: npu_kernel.matmul의 ODS pseudo definition을 완성한다. 최소 operands/results/attrs/verifier 5개를 포함한다.
- 과제 3: npu_rt.launch의 tensor descriptor ABI 필드를 8개 이상 정의하고, compiler가 검증해야 할 항목을 적는다.
- 과제 4: Dialect boundary 별 regression test 이름과 CHECK-LABEL 기준을 설계한다.

17. References

- MLIR Language Reference - <https://mlir.llvm.org/docs/LangRef/>
- Defining Dialects - <https://mlir.llvm.org/docs/DefiningDialects/>
- Creating a Dialect Tutorial - <https://mlir.llvm.org/docs/Tutorials/CreatingADialect/>
- Operation Definition Specification - <https://mlir.llvm.org/docs/DefiningDialects/Operations/>
- Defining Dialect Attributes and Types - <https://mlir.llvm.org/docs/DefiningDialects/AttributesAndTypes/>
- Dialect Conversion - <https://mlir.llvm.org/docs/DialectConversion/>
- Pass Management - <https://mlir.llvm.org/docs/PassManagement/>